PROJECT REPORT

PERSONAL FINANCE
ADVISOR BOT

AI-Powered Financial Management System

Submitted in partial fulfilment of the requirements of the academic project.

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Scope of the Project
- 6. Key Features
- 7. Technology Stack
- 8. System Architecture
- 9. Functional Modules
- 10. Database Design
- 11. Application Workflow
- 12. Installation and Execution
- 13. Configuration
- 14. Testing
- 15. Security and Data Handling
- 16. Limitations
- 17. Future Enhancements
- 18. Conclusion
- 19. References

1. Abstract

Personal Finance Advisor Bot is a web-based financial management application designed to help individuals record income and expenses, plan budgets, monitor savings goals, and understand their financial activity. It provides separate financial modes for students, freelancers, households, and standard users. The application includes an AI-based conversational advisor powered by the Google Gemini API and a built-in rule-based fallback engine. This allows the advisory feature to continue providing basic guidance when a Gemini API key is unavailable or the external service cannot be reached. The system is implemented with Python and Flask, uses SQLite through Flask-SQLAlchemy for data storage, and presents information through responsive web pages, interactive charts, and downloadable CSV reports.

The main purpose of the project is to make everyday financial tracking easier and to convert transaction records into understandable summaries that can support better budgeting and saving habits.

2. Introduction

Managing personal finances requires regular recording of money received, money spent, planned limits, and savings progress. When this information is maintained manually across notebooks or spreadsheets, it can become difficult to review spending patterns and identify areas where expenses can be reduced. Personal Finance Advisor Bot brings these activities together in one web application.

After creating an account, a user can select a financial mode, maintain income and expense records, set monthly category budgets, define savings goals, and review reports. The dashboard presents summaries and alerts, while the AI Advisor can answer questions using the user's financial records and offer general budgeting suggestions.

3. Problem Statement

Many individuals do not have a single, convenient system for recording income, classifying expenses, monitoring budget limits, and tracking savings targets. Manual tracking can make it difficult to understand monthly spending patterns or notice when a category is approaching its limit. A simple financial management system is therefore required to organize this information and provide useful summaries and guidance.

4. Objectives

- Provide a secure user registration and login facility.
- Allow users to record, update, and review income and expense transactions.
- Support financial modes for students, freelancers, households, and standard users.
- Help users create monthly budgets and monitor category-wise utilization.
- Allow users to create savings goals and record contributions.
- Present financial trends through interactive charts and reports.
- Provide conversational financial guidance using Gemini with a rule-based fallback.
- Allow relevant financial records to be exported as CSV files.

5. Scope of the Project

The project focuses on personal financial record keeping and budgeting. It stores information entered by the user and uses those records to calculate summaries, budget utilization, and savings progress. The application is intended for personal organization and educational demonstration. It does not connect to bank accounts or initiate payments, and its generated suggestions should not be treated as professional investment, tax, or legal advice.

6. Key Features

6.1 Specialized Financial Modes

- College Student Mode: supports allowance tracking, weekly spending pace, college-related expense categories, and spending alerts.
- Freelancer Mode: supports variable project income, work-related expense tagging, spending baselines, and emergency-buffer planning.
- Household Mode: supports shared household expense categories such as groceries, rent, utilities, and joint savings targets.
- Standard Mode: provides general personal budgeting, including a 50/30/20-style framework.

6.2 Income and Expense Management

- Income records can include source category, amount, date, description, and recurring status.

- Expense records can include category, amount, date, description, payment method, notes, and work-related status.
- The transaction ledger supports searching, category/date filtering, and sorting.

6.3 Smart Budget Planner

- Generates suggested category limits using income, past spending, and the selected financial mode.
- Shows budget utilization with On Track (below 80%), Warning (80% to below 100%), and Over Budget (100% or more) states.
- Allows a generated budget plan to be applied to the user's records.

6.4 AI Financial Advisor

- Provides a conversational interface for questions about spending and saving.
- Uses Google Gemini through the official Google GenAI Python SDK when configured.
- Uses a rule-based fallback engine when Gemini is not configured or unavailable.
- Provides spending observations, savings suggestions, and a monthly financial health report card.

6.5 Savings Goals

- Allows users to set a goal name, target amount, target date, and description.
- Records contributions made toward a goal.
- Calculates progress, remaining amount, and the monthly contribution required to reach the target.

6.6 Reports and Demo Data

- Displays income-versus-expense trends, expense-category distribution, savings progress, and budget utilization charts.
- Exports transaction, budget, and savings information as CSV files.
- Provides a printable monthly statement.
- Includes a demo-data option to populate sample transactions, budgets, and goals for demonstration.

7. Technology Stack

8. System Architecture

The application follows a modular Flask architecture. The browser displays HTML templates and uses CSS and JavaScript for the interface and charts. Flask receives requests and routes them to blueprint modules. These modules use SQLAlchemy models to read and update the SQLite database, and service modules contain reusable operations for analytics, budget calculations, AI responses, and report generation.

- Presentation layer: Jinja2 templates, Bootstrap, CSS, JavaScript, and Chart.js.
- Routing layer: Flask Blueprints for authentication, dashboard, income, expenses, budgets, savings, reports, and AI advisor.
- Service layer: analytics_service.py, budget_service.py, gemini_service.py, and report_service.py.
- Data layer: SQLAlchemy models connected to an SQLite database.
- External AI service: Google Gemini, with a local rule-based fallback path.

9. Functional Modules

Authentication and User Settings

Handles registration, login, profile settings, financial mode selection, and demo-data seeding.

Dashboard

Displays financial summaries, mode-specific information, notifications, and alerts.

Income Management

Supports adding, editing, listing, and filtering income entries.

Expense Management

Supports adding, editing, listing, and searching expense entries and transaction records.

Budget Management

Creates and manages monthly category limits and shows utilization status.

Savings Management

Maintains savings goals and contribution records.

Reports

Builds financial visualizations, CSV exports, and printable monthly summaries.

AI Advisor

Provides the chatbot interface and coordinates Gemini or fallback responses.

10. Database Design

The database is managed through Flask-SQLAlchemy. The main entities identified in the project models include the following:

The User entity has one-to-many relationships with income, expense, budget, savings goal, AI conversation, and notification records. A savings goal can have multiple contribution records. Budget records enforce uniqueness for a user's category in a particular month and year.

11. Application Workflow

The user opens the application in a web browser.

A new user registers or an existing user signs in.

The user selects or updates a financial mode and currency preference.

The user records income and expenses.

The dashboard and analytics services calculate summaries from the stored records.

The user creates budgets and savings goals and monitors their progress.

The user can ask the AI Advisor questions; Gemini is used when configured, otherwise the fallback engine is used.

The user reviews charts, downloads CSV reports, or prints a monthly statement.

12. Installation and Execution

Prerequisites: Python 3.9 or later, Git, and a web browser.

1. Clone the repository:

```
git clone https://github.com/VedantAppa/personal-finance-advisor-bot.git
cd personal-finance-advisor-bot
```

2. Create and activate a virtual environment (macOS/Linux):

```
python3 -m venv venv
source venv/bin/activate
```

For Windows PowerShell, activate it with:

```
venv\Scripts\Activate.ps1
```

3. Install the Python dependencies:

```
pip install -r requirements.txt
```

4. Create the local environment file from the example:

```
cp .env.example .env
```

5. Start the application:

```
python app.py
```

6. Open http://127.0.0.1:5000 in a browser. The application factory creates the instance directory if required, initializes the database, registers the blueprints, and creates database tables.

13. Configuration

The application reads settings from environment variables. The .env file should be kept local and must not be committed to a public repository.

To enable Gemini, create an API key in Google AI Studio, place it in GEMINI_API_KEY in the local .env file, and restart the Flask application. Never publish the key or include it in screenshots or source control.

14. Testing

The repository includes pytest and a test configuration that uses an in-memory SQLite database and disables CSRF checks for test requests. The README describes a 16-test suite covering authentication, session isolation, income and expense operations, budget limits and calculations, savings goals, contributions, and AI fallback behavior.

Run the tests from the project directory with:

```
pytest -v
```

The test result should be checked in the actual development environment before reporting a pass or including a test-result screenshot in the final submission.

15. Security and Data Handling

- Passwords are stored as hashes using Werkzeug's PBKDF2-SHA256 method rather than as plain text.
- Flask-Login manages authenticated user sessions and protects login-required pages.
- Flask-WTF CSRF protection is initialized for form/request protection.
- Database records are associated with user IDs so that user-specific records can be queried separately.
- Sensitive configuration such as secret keys and Gemini API keys should be stored in a local .env file and excluded from Git.
- Financial data should be backed up carefully; the local SQLite database contains personal information.

16. Limitations

- The application depends on users entering accurate and complete income and expense information.
- Gemini-based responses require an internet connection and a valid API key; fallback responses are rule-based and may be less flexible.
- The project is a personal tracking and educational tool; it does not directly connect to bank accounts, UPI apps, or payment gateways.
- Financial suggestions are general informational guidance and are not a substitute for advice from a qualified financial professional.
- The default setup uses SQLite, which is appropriate for local use and demonstrations but may need a different database and deployment configuration for larger multi-user production use.

17. Future Enhancements

- Add optional bank statement import with user consent and secure file processing.
- Provide recurring transaction reminders and configurable budget notifications.
- Add multi-currency conversion with clearly dated exchange rates.
- Improve mobile usability and accessibility.
- Add stronger production deployment settings, database migrations, backups, and monitoring.
- Expand automated tests and add integration tests for external AI service failures.
- Provide downloadable PDF reports and additional custom report filters.

18. Conclusion

Personal Finance Advisor Bot combines income and expense tracking, budget planning, savings goals, financial reports, and conversational assistance in one Flask-based web application. Its modular structure separates routes, data models, and reusable services, while SQLite provides local data storage. The Gemini integration and rule-based fallback offer two paths for financial assistance. Overall, the project demonstrates how web development, database management, authentication, visualization, and AI integration can be combined to build a practical personal finance tool.

19. References

- Project source code: https://github.com/VedantAppa/personal-finance-advisor-bot
- Flask documentation: https://flask.palletsprojects.com/
- Flask-SQLAlchemy documentation: https://flask-sqlalchemy.palletsprojects.com/
- Flask-Login documentation: https://flask-login.readthedocs.io/
- Google AI Studio: https://aistudio.google.com/
- Chart.js documentation: https://www.chartjs.org/docs/

— End of Project Report —

Submitted by	Vedant Ghodeswar
Project	Personal Finance Advisor Bot
Repository	github.com/VedantAppa/personal-finance-advisor-bot

Submitted by	Vedant Ghodeswar
Academic Year	2026–2027

Layer	Technology
Backend	Python 3.9+, Flask, Werkzeug
Database / ORM	SQLite, Flask-SQLAlchemy
Authentication	Flask-Login; Werkzeug PBKDF2-SHA256 password hashing
Form handling and security	Flask-WTF, CSRF protection, email-validator
AI integration	Google GenAI Python SDK, Gemini 2.5 Flash
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5, Bootstrap Icons
Charts	Chart.js
Configuration	python-dotenv
Testing	pytest

Entity	Purpose
User	Stores user identity, email, password hash, currency, financial mode, and monthly allowance.
Income	Stores each user's income amount, category, description, date, and recurring flag.
Expense	Stores each user's expense amount, category, description, payment method, date, notes, and work-related flag.
Budget	Stores monthly category limits for a user, including month and year.
SavingsGoal	Stores goal name, target amount, current amount, target date, and description.
SavingsContribution	Stores deposits/contributions associated with a savings goal.
AIConversation	Stores AI conversation records associated with a user.
Notification	Stores user notifications and read/unread status.

Variable	Purpose
FLASK_SECRET_KEY	Secret key used by Flask for session/security functions; replace the development default.
GEMINI_API_KEY	Optional Google Gemini API key. Leave blank to use the fallback advisor.
GEMINI_MODEL	Gemini model name; project configuration defaults to gemini-2.5-flash.
DATABASE_URL	Database connection string; project configuration defaults to SQLite.
DEFAULT_CURRENCY	Default currency symbol, configured as ■ unless changed.